

This homework has 9 questions, for a total of 100 points and 10 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework; you may also find the video “walkthroughs” in the Canvas Media Gallery helpful. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn’t show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn’t turn in the previous homework, then (1) state that you didn’t turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: HW4 Question 2a

Solution:

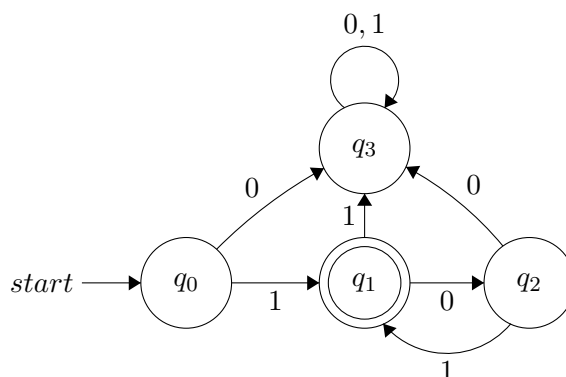
Looking at the function given we know that both a and b are both powers of two as given in the condition of the function. Therefore a good algorithm that combines the aspect of the power of two of both functions which can be done through logarithms. Therefore the potential algorithm can be $P(a, b) = \max(\log_2 a, \log_2 b)$. If we take this function then going from the first case we can see that the max of $\log_2 1, \log_2 1$ or if a and b are equal is going to amount to 0 as neither are greater than or less than each other. For the next case if a is greater than b we can return the max of $\log_2 a/2, \log_2 b$ which becomes $\log_2 a - 1, \log_2 b + 1$ where we can see that since we know a is greater than b the potential decreases by 1. Then if we consider the third case as the max of $\log_2 a, \log_2 b/2$ which becomes $\log_2 a + 1, \log_2 b - 1$ since b is greater than a the potential decreases by 1 here as well. Since in both cases the potential is decreasing and the potential stops at the case when they are both equal then we can see that any input of a or b will have to halt the algorithm at some point and the

algorithm cannot ever run into negative numbers for input.

This solution is incorrect because looking at the given function in the problem, there are many inputs for a and b that are valid even numbers that do not produce any output. For example, $(4,8)$ does not produce any output. This indicates that the function will not terminate for certain cases, which means that there can be no function such that for every valid input of two even numbers the given function always terminates. In order to avoid this mistake, always check test cases before making a determination about a function by theoretical methods such as finding a potential answer to the question first.

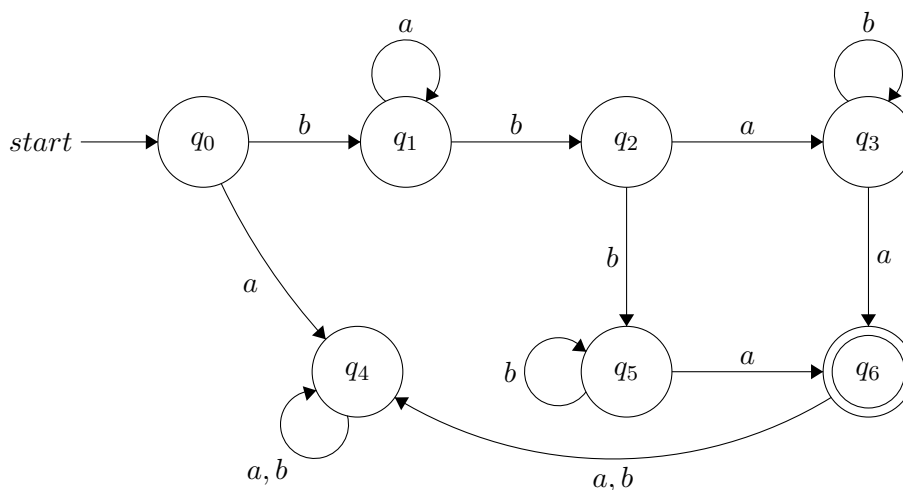
2. Short answer.

- (4 pts) (a) Write the state-transition function (called δ in the lecture and the notes) for the following DFA, as a table. Also give a regular expression (using the “string notation” from class) for the language this DFA decides. No justification is required for either of these. For reference on formatting the state-transition function, see the lecture notes on Finite Automata.



Solution: The state-transition function can be written as $\delta(q0, 0) = q3$, $\delta(q0, 1) = q1$, $\delta(q1, 0) = q2$, $\delta(q2, 1) = q1$, $\delta(q1, 1) = q3$, $\delta(q2, 0) = q3$, $\delta(q3, 0) = q3$, $\delta(q3, 1) = q3$. Regular expression: $101 + 0(10)^* + 11(10)^*$

- (4 pts) (b) Give a regular expression (using the “string notation” from class) for the language decided by the following DFA. No justification is required.



Solution: Regular expression: $ba^*bab^*a(ab)^*bb^*a$

(4 pts)

- (c) Consider the following claim: a minimum spanning tree in a graph is *unique* if the edge weights are *distinct* (all different).

Briefly explain what is wrong with the following bogus “proof” of this claim: *because edge weights are distinct, all choices in Kruskal’s algorithm are completely determined—there are no “ties” between edge weights that the algorithm can choose how to break. So, the algorithm has only one possible output. Because Kruskal’s algorithm is correct for the MST problem, its unique output must be the only minimum spanning tree in the graph.*

Solution: The claim made is incorrect because it assumes that there is a unique choice between each possible aspect of the graph. It does not correctly assume that there are many ways to continue on a tree in the output and that different MSTs resulting from different spanning trees that may have equal weight regardless of edge case weights because there is more than one way to connect a tree through different edges that can add up to the same minimum spanning tree but a different construction.

(4 pts)

- (d) Give a correct proof for the claim in the previous part.

Solution: Using a proof by contradiction, we can firstly claim that a MST in a graph with distinct weight on edges is unique. Then we can assume there exist two spanning trees in a graph with distinct edges that are minimum spanning trees. If we think this is incorrect, to see this we can take the smallest weighed edge and add it to one tree. If the two trees are different, then the other tree contains another edge that adds up to the minimum value. Since the edges are distinct, this cannot be possible, therefore there is no way to have two unique MSTs in one same distinct edge graph.

3. Greedy knapsack filling.

Recall that in the “0-1” knapsack problem, you are given n items, where the i th item has weight w_i and value v_i (both positive), and a non-negative weight capacity C of the knapsack.

An optimal solution is a subset $S \subseteq \{1, 2, \dots, n\}$ of the items having maximum total value

$$\sum_{i \in S} v_i ,$$

under the constraint that the total weight is at most the knapsack capacity, i.e.,

$$\sum_{i \in S} w_i \leq C .$$

In this problem, we introduce the *fractional* variant of the knapsack problem, where one may take any fraction $p_i \in [0, 1]$ of any item i . Naturally, a p_i -fraction of item i weighs $p_i \cdot w_i$, and has value $p_i \cdot v_i$. The goal is to maximize the total value of the selected fractional items.

- (3 pts) (a) Briefly explain why the optimal value for the fractional knapsack problem is *at least as large* as that of the original 0-1 knapsack problem (for the same weights, values, and knapsack capacity).

Solution: In the fractional knapsack problem there is the possibility of taking just fractions of items instead of only being able to take whole item values. This means that if we have an item that weighs more than what the capacity can hold, a fraction of that item can be taken and included in the 0-1 knapsack problem. Therefore we know that the optimal value of the fractional knapsack problem can be the same as the normal problem because we would just keep the fraction of each item taken to 0 or 1. We also know that taking the fraction of larger items that may exceed the capacity will allow them to fit under the capacity, so items from the normal problem that were excluded with a 0 factor can instead be multiplied by a fractional factor to increase the optimal total value.

- (4 pts) (b) Consider the following two greedy algorithms for the fractional knapsack problem:
- Algorithm *A*: While there is still some unused knapsack capacity, choose an item having the *largest value* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.
 - Algorithm *B*: While there is still some unused knapsack capacity, choose an item having the *smallest weight* (among those not considered yet), and add the largest possible fraction of that item (up to the entire item) that will fit within the remaining knapsack capacity.

Suppose we run *both* algorithms *A* and *B*, and output a best one of their two outputs. Is this a correct algorithm for the fractional knapsack problem? If so, prove it. Otherwise, give an input for which this algorithm's output is not optimal, and show why it is not.

Solution: Considering algorithm *A*, we can see that the algorithm first chooses the maximum value and either fractions it to fit the capacity or enters it whole until another item can make the capacity be reached. In this algorithm, we can see that the threshold will either always be reached or all items will be used at full weight and the

threshold of capacity is higher than all items combined. However, we can also see that this is not an ideal solution for the knapsack problem. To specify with an example, if we only choose by maximum value, then if the weight capacity is 5 and there are items such that 3 items have weight 1, 2, and 3 and values 4,5, and 6 while the fourth item has weight 5 but value 7, then the algorithm will output the max value as 7 when the optimal max value can be 11. Therefore this does not work. Considering algorithm B, we can use similar logic to see that minimum weights being added first also do not solve the problem. If there are three items such that weights are 2,3, and 5 and values are 4,6, and 11 then if the capacity is a weight of 5, this method will keep the max value to be 10, while the optimal max value in this situation can be 11. Therefore this algorithm also does not yield the true best output. Since neither algorithm yields the optimal output the joint running will not either.

(7 pts)

(c) Consider the following greedy algorithm for the fractional knapsack problem:

- Algorithm *C*: While there is still some unused knapsack capacity, choose an item having the largest *relative value* v_i/w_i (among those not considered yet), and add the largest possible fraction of that item (up to the full item) that will fit within the remaining knapsack capacity.

Here is the skeleton of an inductive proof that Algorithm *C* outputs an optimal solution. Without loss of generality, suppose that the items are in sorted order by relative value $r_i = v_i/w_i$, from largest to smallest (the algorithm considers them in this order). View any selection of fractional items as a sequence $p_1, p_2, \dots, p_n$, where p_i is the fraction of item i . Let $g_1, g_2, \dots, g_n$ be the sequence of fractions that the greedy algorithm selects. We aim to prove the following claim:

For *every* $k = 0, \dots, n$, the first k values $g_1, \dots, g_k$ of this sequence are a prefix of (i.e., are the first k item fractions in) *some* optimal solution.

Given this claim, the full sequence (for $k = n$) is itself an optimal solution, because there are no more items that can be (fractionally) selected.

Prove the claim by induction. Specifically, establish the base case $k = 0$, and then prove the inductive step, which says: for any $k < n$, if $g_1, \dots, g_k$ is a prefix of *some* optimal solution $OPT = o_1, \dots, o_n$, then $g_1, \dots, g_k, g_{k+1}$ is a prefix of *some* optimal solution $OPT' = o'_1, \dots, o'_n$. Use an exchange argument to modify OPT into a valid selection OPT' , without reducing the value (so that it remains optimal).

Solution: Proof by induction:

Base case:

If $k = 0$ then the first 0 values $g_1, \dots, g_0$ of this sequence are a prefix of some optimal solution $OPT' = o'_1, \dots, o'_0$, and in this case choosing 0 items is the optimal solution to a desired optimal value of 0.

Inductive step:

Assume that for some $k < n$, the first k values of this sequenced $g_1, \dots, g_k$ are a prefix of some optimal solution $OPT' = o'_1, \dots, o'_k$. Next, we can see from algorithm C that the largest relative value is added if there is unused knapsack capacity. Therefore

Homework 5

we can say that for a case where we have $g_1, \dots, g_{k+1}$ are a prefix of some optimal solution, then if there is an optimal solution to this such that $OPT' = o'_1, \dots, o'_{k+1}$, since g_{k+1} is the largest relative value added to the list from $g_1, \dots, g_k$, we can rewrite the expression for $k + 1$ as $OPT' = o'_1, \dots, o'_k, g'_{k+1}$ which is the exchange argument. Therefore we can write this expression as $OPT'(k + 1) = OPT'(k), g'_{k+1}$. Therefore for k from 1 to n , we can see that the first k values are the prefix in some optimal solution to the problem.

- (8 pts) (d) Now consider a new variant of the fractional knapsack problem, in which there are ℓ knapsacks, where the j th knapsack has capacity C_j , and we aim to maximize the total value across all the knapsacks. Give a greedy algorithm that solves this variant.

Solution: Algorithm D: First sort the items with the largest relative value v_i/w_i first and the least values last and initialize a container to place all potential items for each knapsack in ℓ knapsacks. Then find the knapsack with the least remaining capacity and add the largest relative value or a fraction that is the largest that can fit of that value to the knapsack. Continue the process of selecting the item and knapsack to place the item until either no items are left or all knapsacks have reached capacity.

4. Greedy touring.

PETROBLUE, in support of the M Bus Formula One Team, has innovated a modular fuel pack for a bus that will transport EECS 376 students on a tour through various towns in Michigan. This fuel pack allows the vehicle to travel a certain distance before refueling. Each town has a service station where a fuel pack can be reloaded: the current pack is simply swapped out for a fresh full one. (Any remaining fuel in the removed pack will not be used by the bus.) The goal is to visit the towns in a specified order, while minimizing the number of fuel pack reloads.

We model this problem as follows. The bus can travel a distance up to R on a single fuel pack. There are n towns to visit in sequence. For $i = 1, \dots, n$, the distance from the $(i - 1)$ st town (or from the starting point, when $i = 1$) to the i th town is $d_i \leq R$. Given R and an array $D[1, \dots, n] = [d_1, \dots, d_n]$ as input, the goal is to find a *smallest* set $S \subseteq \{1, \dots, n\}$ of towns (specified by their indices) where refueling should occur, so that the bus can reach all n towns without running out of fuel at any point. The bus starts out with a full fuel pack, and no refueling is needed at the final town.

- (8 pts) (a) Give a greedy algorithm (including pseudocode) that solves this problem. Defer a correctness argument to the next part, but provide everything else involved in “giving an algorithm” here.

Solution: Algorithm: While there are still towns in Michigan to visit find the farthest town that can be visited without refueling from the current bus position and add it to the list of S towns needed to fuel at.

Pseudocode:
function($R, D[1, \dots, n]$)
 S = set of towns

```
position = 0 (start town)
while position != size(D) - 1
  farthest = position
  for(int i = position; i < size(D); ++i)
    if(D[i] < R)
      farthest = i
  end for loop
  add farthest to the set S
  position = farthest

return S
```

- (8 pts) (b) Give an inductive proof using an exchange argument (like the ones presented in lecture and in the previous problem) that your algorithm from the previous part is correct.

Solution: Inductive proof:

Base case:

When $k = 0$ then the farthest reachable town from the current position is not of any distance. Therefore since the last position does not require any fueling the base case is true because the bus never runs out of fuel.

Inductive step:

Assume that for a $k < n$ there is an optimal set of towns S such that the bus can travel to k towns without running out of fuel through refueling stops. Therefore we can say that if there is a set S that is the optimal set of towns to travel k towns, then $S1$ is the optimal set of towns to travel $k + 1$ towns such that S is a subset of $S1$. Therefore if S is the optimal solution of k towns, we can apply the exchange argument by setting all towns in S to all towns in $S1$ by noting that if the last town to travel to in S is the farthest reachable town from the previous destination, we can exchange the towns that are closest to the towns in S with towns that are also in $S1$ such that $S1-S$ is the resulting set of $k+1$ where the set is a prefix of some optimal solution for reaching the $k+1$ town without running out of fuel.

- (4 pts) (c) Now, suppose that the price of a fuel pack may differ from town to town. For example, a fuel pack might cost \$8 in the first town, \$3 in the second town, etc. So, the input now also includes an array $C = [c_1, \dots, c_n]$, where c_i is the cost of refueling in the i th town. The goal now is to complete the tour while *minimizing the total cost* of the fuel packs that are purchased along the way.

PETROBLUE claims that we should use the same greedy algorithm that you proposed above to solve this problem. Determine whether this claim is true or not. If it is, give a proof. Otherwise, provide a small counterexample: give a specific input, the output of your greedy algorithm on that input, and a better solution for that input; also, *very briefly* explain where your proof from the previous part fails for this problem.

Solution: This solution to the problem may not work with the new price difference in refueling. The algorithm above only takes into account distance, which means that the greedy algorithm only goes to the farthest town and doesn't take into account the cost of fuel, therefore the most expensive fueling site could be chosen instead. If we take an example where R is 7, there are three towns from the current location with distances 3,4,3 and costs 2,15,15. If we go by the greedy algorithm we can see that the choice would be to stop at town 2 first because 7 distance can be covered by the fuel back, but the cost at that location is much higher than if refueling occurred in the first town and the second town was skipped in refueling. Therefore for this input we would want the cost to be considered first and therefore the algorithm should choose towns that cost the least to fuel over the distance instead. The last program fails because the cost is not taken into account when choosing the route, so we end up with a not optimal solution for the cost.

(3 EC pts)

- (d) *Optional extra credit.* Give an efficient dynamic programming algorithm (with recurrence and bottom-up implementation) that solves the problem from the previous part.

Solution: To give a dynamic programming algorithm, we can create an array that keeps track of both the minimum cost to reach a town and the distance remaining to reach the town. Therefore the array can be represented as $dp[i][j]$ where cost and distance are the key values. Initialize $dp[0][0]$ to 0, as the bus starts at town 0 with a full fuel pack. For each i from 0 to R , and each j from 0 to $n - 1$ Initialize $dp[i][j+1]$ to $dp[i][j]$ to represent not refueling at town $j+1$. For each k from 0 to i , calculate $dp[i-k][j+1]$ considering refueling at town $j+1$. Update $dp[i][j+1]$ with the minimum cost. The optimal solution is given by $dp[R][n]$, representing the minimum cost to reach the last town with the entire distance covered.

5. Selecting suitable skis.

Suppose there are n skiers of various heights, and n pairs of skis having various lengths. (Both skis in a pair have the same length.) We want to match each skier with a pair of skis so as *minimize the total difference* between each skier's height and the length of their skis.

Formally, we are given a *sorted* array $P[1, \dots, n]$ of the skiers' heights (where $P[i] \leq P[i + 1]$ for all $i < n$), and a (not necessarily sorted) array $S[1, \dots, n]$ of the skis' lengths. We want to return a rearrangement (permutation) $S'[1, \dots, n]$ of S that minimizes

$$\sum_{i=1}^n |P[i] - S'[i]|.$$

(3 pts)

- (a) Give an $O(1)$ -time algorithm for the case $n = 2$.

Solution: For the case $n = 2$ we can see that there are only two ways to arrange the skis since n is only 2. Therefore we can write an algorithm to compute the values of each pairing and the minimum height of each pairing. First compute $P[2] - S[2]$, then

$P[2] - S[1]$, then $P[1] - S[2]$, and $P[1] - S[1]$. After computing all four possible values of height return the minimum height found of the four computations by adding the correct two pairings together and finding the minimum total height. This is an $O(1)$ algorithm as it operates in constant time with direct calculation.

- (7 pts) (b) For general n , consider an algorithm that just returns S in sorted order. We can view this as a greedy algorithm that just repeatedly selects a pair of skis of shortest length (among those that remain). So, it makes exactly n selections $s'_1, s'_2, \dots, s'_n$ in sorted order. Give an inductive proof using an exchange argument (like the ones presented in lecture and in the previous problems) that this algorithm is correct.
Hint: Use the previous part as an ingredient of your proof.

Solution: Inductive Proof:

Base case:

Since i only goes from 1 to n , we can set the base case to be $n = 1$ where this is the only pair of skis available and only one skier, so there is only one solution to the minimum height which is $P[1] - S[1]$.

Inductive step:

Assume that the algorithm holds true for $n-1$ skiers and skis such that there is a permutation of skiers and skis such that there does exist a minimum height sum of the skiers and their ski length differences. Let S' be the permutation for $n-1$ skiers. Then let there be a set of sorted skis and skiers from height and length in $n-1$. Now to add the n th skier and ski, sort them by height and length into the sets. Now let S'' be the permutation for n skiers. Now, we consider the permutation obtained by the algorithm for n skiers, denoted as algorithm, where we repeatedly select the pair of skis with the shortest length.

Exchange Argument: Let sorted $[i]$ be the i -th pair of skis in the sorted order, and algorithm $[i]$ be the i -th pair of skis obtained by the algorithm.

Consider the exchange of algorithm $[i]$ and S sorted $[i]$. We want to show that this exchange does not increase the total difference in heights. Let OPT be the permutation obtained by exchanging algorithm S algorithm $[i]$ and sorted S sorted $[i]$ in OPT . We have two cases: algorithm S algorithm $[i]$ is selected in the same position in OPT as sorted S sorted $[i]$: In this case, the total difference remains unchanged because the skier's height is matched with the pair of skis with the shortest length. S algorithm $[i]$ is selected in a different position in OPT from sorted S sorted $[i]$: In this case, we know that algorithm $[i]$ must be selected at a position that is farther away from the skier's height, compared to the position where S sorted $[i]$ is selected. Therefore, exchanging them will decrease the total difference.

Since exchanging pairs in OPT either keeps the total difference the same or reduces it, we conclude that the algorithm remains correct when adding the n -th skier.

By induction, the algorithm is correct for any number of skiers n .

6. Decision problems and languages.

Homework 5

Recall that the goal of a *decision problem* is to determine whether a given input “object” has a certain “property”, e.g., determine whether a given integer is prime, or whether a given string is a palindrome. In class, we said that any decision problem is equivalent to a *membership problem* for a corresponding *language*. That is, determining whether a given object has the property is equivalent to determining whether its encoding (as a string) is a member of the language.

For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, (iii) analyze the length of the encoding in terms of the value of the input, and (iv) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet. As examples, we have provided solutions to the first two parts.

- (0 pts) (a) “Does a given non-negative integer k have 3 as its last digit, when written in base 10?”

Solution:

- (i) A reasonable alphabet consists of the decimal digits, $\Sigma = \{0, \dots, 9\}$.
- (ii) Encoding: for an integer k , write it in base 10 in the usual way, as a string of digits. For example, if k is the integer forty-seven, then $\langle k \rangle = 47$. We stress that this is a *string* of the characters 4 and 7, which *represents* the number forty-seven.
- (iii) The length of this encoding would be the number of digits in the value, which is $\Theta(\log k)$.
- (iv) A corresponding language would be $L_{EndsWith3} = \{\langle k \rangle : k \bmod 10 = 3\}$. It would also be acceptable to copy the phrasing of the decision problem, i.e., $L_{EndsWith3} = \{\langle k \rangle : k \text{ written in base 10 has 3 as the last digit}\}$

Alternatively, we could also encode integers in binary, using the alphabet $\Sigma = \{0, 1\}$. For example, if $k = 5$, then $\langle k \rangle = 101$. The length of this encoding is still $\Theta(\log k)$. The above two definitions of the language hold without modification, because they both describe membership only in terms of the *value* of k , not its encoding.

- (0 pts) (b) “Is a given array of non-negative integers sorted?”

Solution:

- (i) A reasonable alphabet Σ is the set of ASCII characters, or more selectively, the decimal digits along with some separator characters: $\Sigma = \{0, \dots, 9, [, ,,]\}$. (Notice that a comma is one of these characters.)
Note: we can’t just use the decimal digits alone if we want to use some special symbols to indicate the start/end of the array and to separate the elements.
- (ii) Example encoding: For an array A , encode its elements as in the previous part, and list those encodings with appropriate separators. For example, the array A with entries one, two, three, and four would have $\langle A \rangle = [1, 3, 2, 4]$.

(iii) The length of this encoding is $\Theta(n + e)$, where n is the number of elements in the array, and e is the total length of the encoding of the elements.

(iv) The corresponding language is

$$L_{sorted} = \{\langle A \rangle : A \text{ is a sorted array of non-negative integers.}\}.$$

(3 pts) (c) “Given an array S of integers, are all its elements distinct?”

Solution:

(i) A reasonable alphabet Σ is the set of ASCII characters, or more selectively, the decimal digits along with some separator characters: $\Sigma = \{0, \dots, 9, [, ,,]\}$.

(ii) Example Encoding: for an array S , write it such that the elements are encoded like in the previous part and list those encodings with separators. For example, the array S with entries one, two, three, and four would have $\langle S \rangle = [1, 3, 2, 4]$.

(iii) The length of this encoding is $\Theta(n + e)$, where n is the number of elements in the array, and e is the total length of the encoding of the elements..

(iv) A corresponding language would be

$$L_{distinct_elements} = \{\langle S \rangle : S \text{ array where elements are integers and all are distinct.}\}.$$

(3 pts) (d) “Is a given undirected graph complete?” *Hint: Consider the adjacency matrix.*

Solution:

(i) A reasonable alphabet Σ is the set of ASCII values or the set of binary values of 0 and 1 in an adjacency matrix.

(ii) Example Encoding: for an undirected graph G , it can be represented with an adjacency matrix where each entry is encoded in the matrix as a binary value or an ASCII value. If G has two vertices it can be represented as $G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

(iii) The length of this encoding is $\Theta(n^2)$, where n is the number of vertices in the graph which need to be added along two dimensions as is required of a matrix.

(iv) A corresponding language would be

$$L_{undirected_complete} = \{\langle G \rangle : G \text{ is a undirected complete graph}\}.$$

(3 pts) (e) “Does a given C++ program halt when run on a given input?”

Solution:

- (i) A reasonable alphabet Σ is the set of ASCII characters used in a C++ program.
- (ii) Example encoding: For an input we can represent the running files as strings entered into the command line. For example, a program with a cpp file and an input file would be entered as $\langle main.cpp, input.txt \rangle$
- (iii) The length of this encoding is $\Theta(n+e)$, where n is the length of the cpp program file and e is the length of the input file.
- (iv) The corresponding language is

$$L_{C++programhalt} = \{\langle cpp, txt \rangle\} : \text{for the input given the } C++ \text{ program halts.}$$

7. Deciding languages with DFAs.

For each of the following languages, give a DFA that decides it. You may represent a DFA as a state-transition function or as a diagram, whichever you prefer.

- (3 pts) (a) The language (over alphabet $\{a, b\}$) corresponding to the regular expression ab^* .

Solution: As a state-transition function:

At state q_0 the language must start with a and be followed by b. There for q_0 is a and q_1 can be represented as b. Therefore we can represent the language as the function $\delta(q_0, a) = q_1$ and $\delta(q_1, b) = \text{accepting state}$.

- (5 pts) (b) The language (over alphabet $\{a, b\}$) corresponding to the regular expression $b(aaa)^*b$.

Solution: Since we have a starting of b followed by exactly 3 a's and then any number of a's until there must be a b at the end, we can represent this as $\delta(q_0, b) = q_1$, $\delta(q_1, a) = q_2$, $\delta(q_2, a) = q_3$, $\delta(q_3, a) = q_2$, and $\delta(q_3, b) = \text{accepting state}$.

- (5 pts) (c) The language of binary strings in which the number of 0s is divisible by 3 and the number of 1s is even.

For example, the strings ϵ , 000, 01010, and 11000 are in this language, while 1, 01, 001, 0011, and 000111 are not.

Solution: Since we have binary strings where we can only have the number of 0s be divisible by three and the number of 1s be even, we can have the first three entries be only 0s. We can also have the first three entries be ones. If the ones are true then the next entries must be 0. If the 0s are true then the other entries should include ones. $\delta(q_0, 0) = q_1$, $\delta(q_1, 0) = q_2$, $\delta(q_2, 0) = q_0$, $\delta(q_0, 1) = q_3$, $\delta(q_1, 1) = q_4$, $\delta(q_2, 1) = q_3$, $\delta(q_3, 0) = q_4$, $\delta(q_4, 0) = q_1$, $\delta(q_3, 1) = q_2$, $\delta(q_4, 1) = q_1$

- (4 EC pts) (d) *Optional extra credit.*

Consider the following language:

$$L = \{x \in \{0, 1\}^* : [x] \bmod 3 = 0\}.$$

where $[x] := \sum_{i=0}^n 2^i x_i$ for $x = x_0 x_1 \dots x_n$. E.g., $[\varepsilon] = 0$ and $[01100000] = [011] = 6$. Note that the bits are given in ‘reverse’ order from what is typical (e.g., 6 in binary is usually written as 110).

Give a DFA that decides the language L .

Hint: Relate $[x_0 x_1 \dots x_k]$ to $[x_0 x_1 \dots x_{k-1}]$, x_k , and k . You can use the fact (which is easy to prove) that $2^{2m} \bmod 3 = 1$ and $2^{2m+1} \bmod 3 = 2$ for any non-negative integer m .

Solution:

(3 EC pts) 8. *Optional extra credit.*

Consider the alphabet $\Sigma = \{ (,) \}$, which consists of the open- and close-parenthesis characters (and). Say that a string $x \in \Sigma^*$ is *balanced* if x has an equal number of (and) characters, and every *prefix* of x has at least as many (characters as) ones.

For example, the empty string ε , $((()))$, and $((()))$ are balanced, but $((()))$ is not, because it fails the second condition (but not the first one).

Either give a DFA that decides the language of balanced strings over Σ , or rigorously prove that no such DFA exists.

Solution: